

TALLER #10 –AVANCE 2 DEL PROYECTO

CARLOS EDUARDO BEDOYA VALENCIA – 2230336
MARÍA PAULINA GARCÍA HERNÁNDEZ – 2233132
HECTOR DABID MUÑOZ MUÑOZ - 2233170

Plataforma social - implementación a nivel de Backend y Base de Datos MongoDB

DOCENTE

KAREN VANESSA LOZANO MANRIQUE

UNIVERSIDAD AUTONOMA DE OCCIDENTE
FACULTAD DE INGENIERÍA
ESTRUCTURAS DE DATOS Y ALGORITMOS 2
CALI – VALLE
2026

Trabajo Escrito: Plataforma Social - implementación a nivel de Backend y Base de Datos MongoDB

Introducción

El presente trabajo tiene como propósito documentar el diseño e implementación del lado del servidor (backend) y la integración con la base de datos de una plataforma social. En esta segunda fase del proyecto, se busca evidenciar la construcción de una arquitectura robusta basada en servicios web, permitiendo la gestión eficiente de la lógica del negocio, la comunicación con el cliente (frontend) y la persistencia de la información.

A diferencia del primer avance, centrado en la interacción del usuario y la manipulación del DOM, este desarrollo aborda la estructuración del sistema desde el servidor, incorporando el uso de tecnologías como Node.js, la implementación de rutas, controladores, middlewares y modelos, así como la conexión con una base de datos NoSQL (MongoDB).

Asimismo, se integran mecanismos de comunicación en tiempo real mediante sockets, lo cual permite simular comportamientos propios de una red social moderna, como mensajería instantánea y notificaciones dinámicas. De esta manera, se logra una solución más completa, escalable y alineada con arquitecturas reales de aplicaciones web.

Implementación del Backend y Servicios Web

Para el desarrollo del backend se adoptó una arquitectura modular basada en el patrón Modelo–Vista–Controlador (MVC), adaptado a servicios web RESTful. Esta estructura permite organizar el código en componentes independientes, facilitando su mantenimiento, escalabilidad y reutilización.

Estructura del proyecto

El sistema fue organizado en diferentes carpetas, cada una con una responsabilidad específica:

- **controllers/**: Contiene la lógica del negocio. Aquí se procesan las solicitudes del cliente, como autenticación, gestión de usuarios, publicaciones, amigos y mensajes.
- **models/**: Define los esquemas de datos mediante MongoDB, representando entidades como usuarios, publicaciones y mensajes.
- **routes/**: Expone los endpoints de la API REST, permitiendo la comunicación entre el frontend y el backend.
- **middlewares/**: Implementa funciones intermedias como validación de autenticación y manejo de errores.
- **sockets/**: Gestiona la comunicación en tiempo real para funcionalidades como chat y notificaciones.

- **utils/**: Incluye utilidades y estructuras auxiliares como colas de notificaciones y manejo de feeds.
- **data/**: Contiene datos de prueba en formato JSON utilizados para inicialización o pruebas del sistema.

Esta organización permite separar claramente las responsabilidades dentro del sistema, siguiendo principios de diseño limpio.

Gestión de solicitudes y lógica del sistema

El backend funciona mediante el uso de endpoints HTTP que permiten realizar operaciones CRUD (Crear, Leer, Actualizar y Eliminar). Cada solicitud es procesada siguiendo un flujo estructurado:

1. El cliente realiza una petición a una ruta específica.
2. La ruta dirige la solicitud al controlador correspondiente.
3. El controlador ejecuta la lógica necesaria.
4. Se interactúa con la base de datos mediante los modelos.
5. Se retorna una respuesta al cliente en formato JSON.

Por ejemplo, acciones como registrar un usuario, iniciar sesión, crear una publicación o enviar un mensaje siguen este flujo, garantizando consistencia en el sistema.

Implementación de Middlewares

Los middlewares cumplen un papel fundamental dentro de la aplicación, ya que permiten interceptar las solicitudes antes de llegar a los controladores.

Entre sus funciones principales se encuentran:

- **Autenticación**: Verifica la identidad del usuario antes de permitir el acceso a rutas protegidas.
- **Manejo de errores**: Centraliza la gestión de excepciones, evitando fallos inesperados en la aplicación.

Este enfoque mejora la seguridad del sistema y facilita la depuración de errores.

Integración con Base de Datos (MongoDB)

Para la persistencia de los datos se utilizó MongoDB, una base de datos NoSQL orientada a documentos. Esta permite almacenar información en formato JSON, lo cual resulta ideal para aplicaciones web modernas.

Se definieron modelos para las principales entidades del sistema:

- **Usuario (User):** Almacena información como nombre, correo y credenciales.
- **Publicación (Post):** Representa el contenido compartido por los usuarios.
- **Mensaje (Message):** Gestiona la comunicación entre usuarios.

La conexión con la base de datos se realiza desde el servidor, permitiendo operaciones eficientes y escalables. Además, se implementaron scripts de inicialización (seed.js) para poblar la base de datos con información de prueba.

Comunicación en Tiempo Real (Sockets)

Una de las mejoras más significativas en esta fase es la implementación de sockets, los cuales permiten la comunicación bidireccional entre cliente y servidor en tiempo real.

Gracias a esto, se pueden manejar funcionalidades como:

- Envío y recepción instantánea de mensajes.
- Actualización dinámica de notificaciones.
- Interacción en tiempo real entre usuarios.

Esto acerca la aplicación al comportamiento de plataformas sociales reales.

Uso de estructuras de datos en el Backend

Al igual que en el frontend, en el backend se aplican estructuras de datos para optimizar el rendimiento:

- **Colas (Queues):** Utilizadas en la gestión de notificaciones.
- **Listas y arreglos:** Para manejar publicaciones y relaciones entre usuarios.
- **Estructuras personalizadas:** Implementadas en utilidades como `activityStack.js` y `notificationQueue.js`.

Estas estructuras permiten gestionar la información de manera eficiente y garantizan un buen rendimiento en la ejecución de operaciones.

Soporte de evidencias

A continuación, se presentan las capturas de pantalla correspondientes a las pruebas funcionales y técnicas realizadas sobre el backend de la plataforma social. Estas evidencias permiten validar el correcto funcionamiento de los servicios implementados, así como la adecuada integración entre los diferentes componentes del sistema.

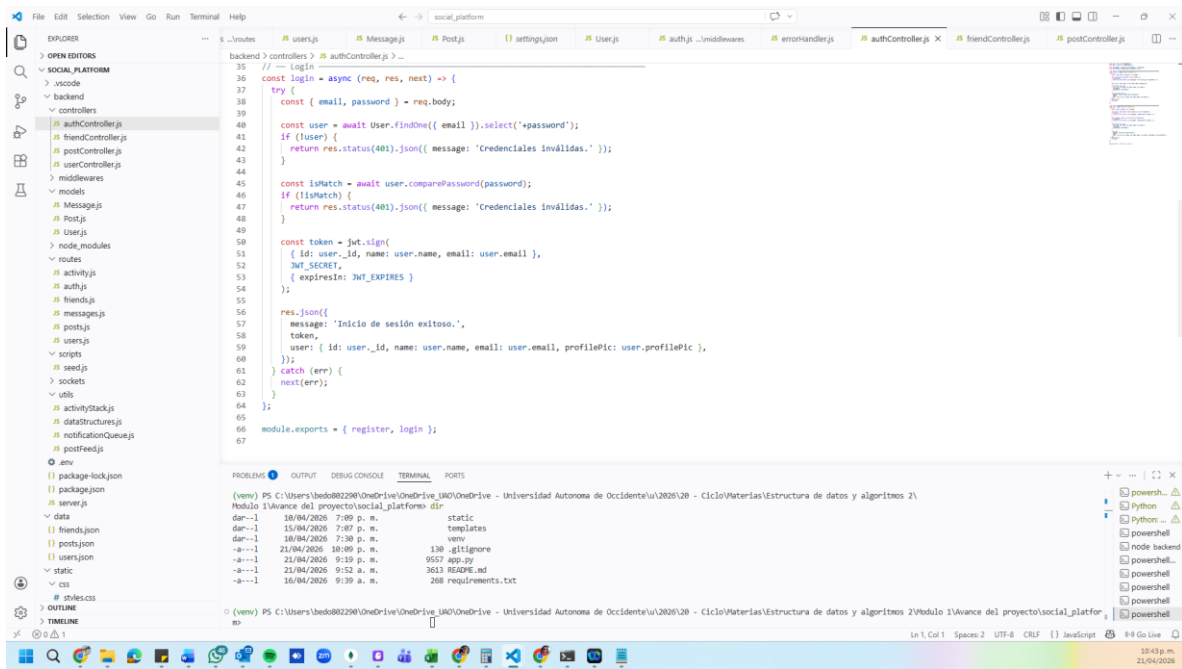
En este avance, las pruebas no se limitan a la interacción visual del sistema, sino que se enfocan en la ejecución del servidor, la respuesta de los endpoints y la persistencia de la información en la base de datos. Por esta razón, se incluyen capturas que evidencian el comportamiento del sistema en tiempo de ejecución, junto con la verificación directa de los datos almacenados.

De manera específica, se presentan imágenes del programa en funcionamiento, donde se observa el procesamiento de solicitudes y la correcta respuesta del backend ante distintas operaciones, tales como registro de usuarios, autenticación, creación de publicaciones y gestión de interacciones.

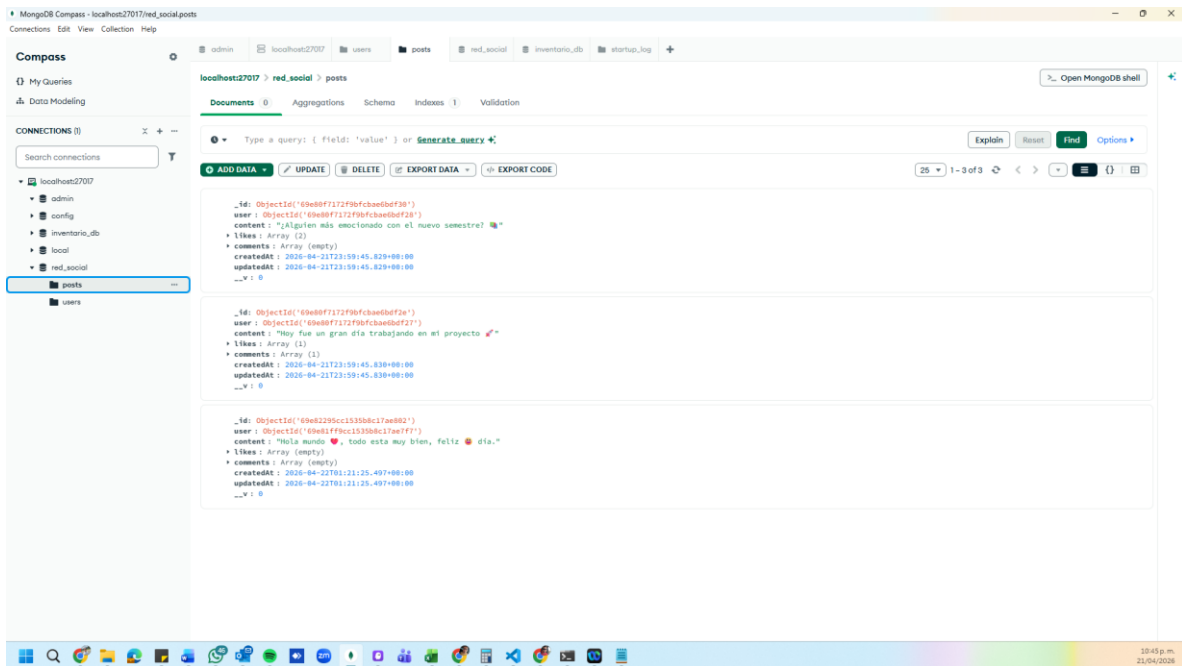
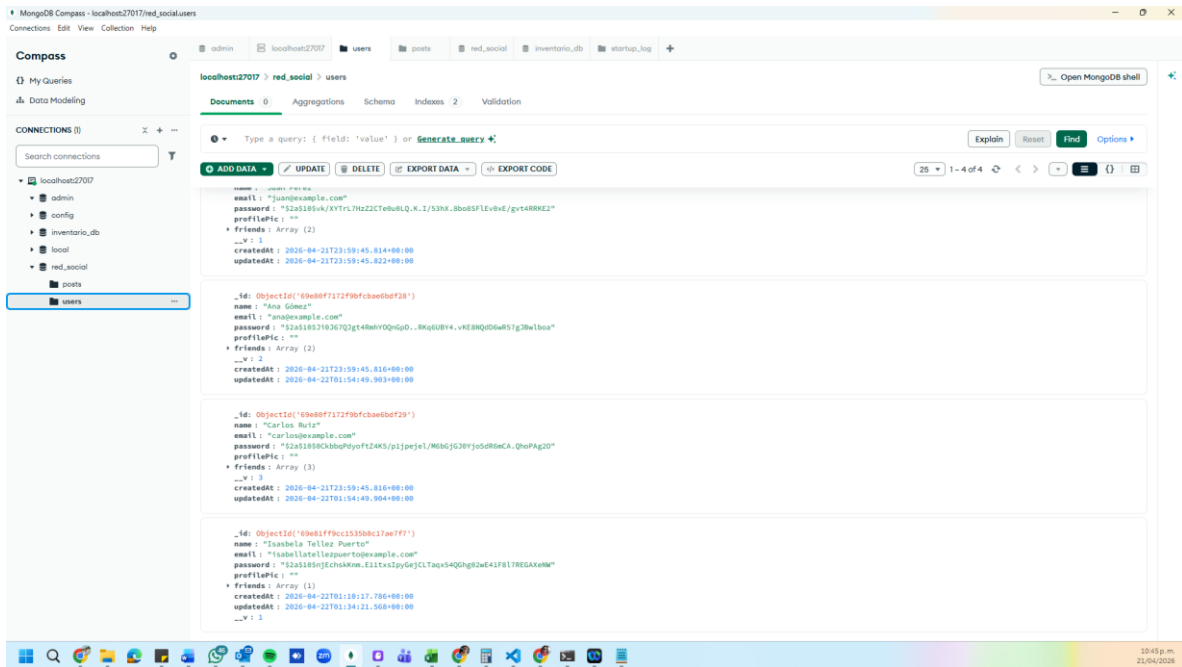
Adicionalmente, se incluyen capturas de la base de datos MongoDB, en las cuales se evidencia la creación y actualización de documentos dentro de las colecciones correspondientes. Esto permite comprobar la persistencia de los datos y la coherencia entre las operaciones realizadas desde el sistema y su almacenamiento.

En conjunto, estas evidencias respaldan la correcta implementación del backend y demuestran que la plataforma cumple con los requisitos establecidos para esta fase del proyecto.

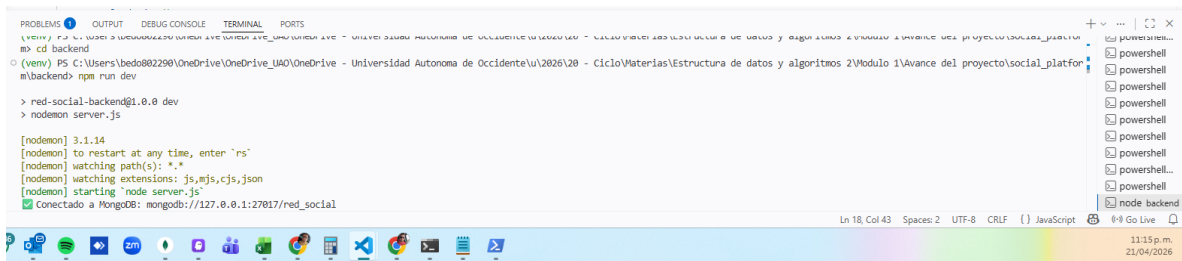
Montaje del backend



Montaje y funcionamiento de la base de datos:



Conexión al servidor de Mongo:

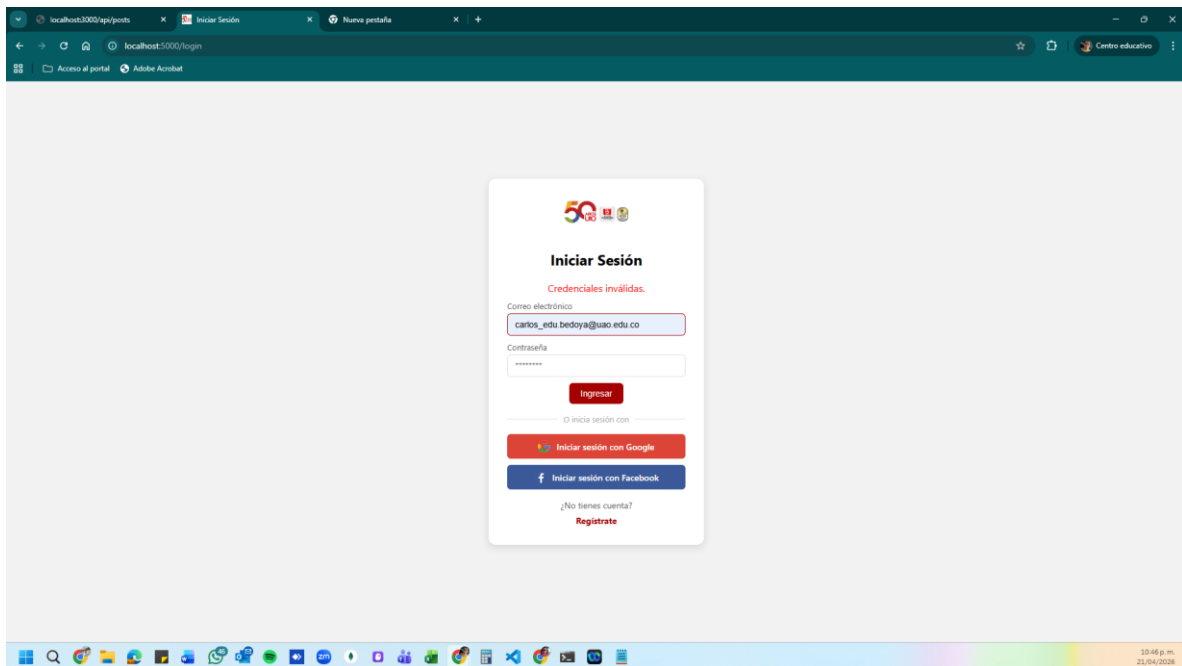


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(venv) PS C:\Users\bed0802290\OneDrive\OneDrive_UAO\OneDrive - Universidad Autonoma de Occidente\2026\20 - Ciclo\materias\Estructura de datos y algoritmos 2\Modulo 1\Avance del proyecto\sociai_platfor
m\backend> npm run dev

> red-social-backend@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node server.js'
[Conectado a MongoDB: mongodb://127.0.0.1:27017/red_social]
```

Pruebas de inicio de sección con un perfil no creado, sin poder iniciar, porque no esta en la base de datos:



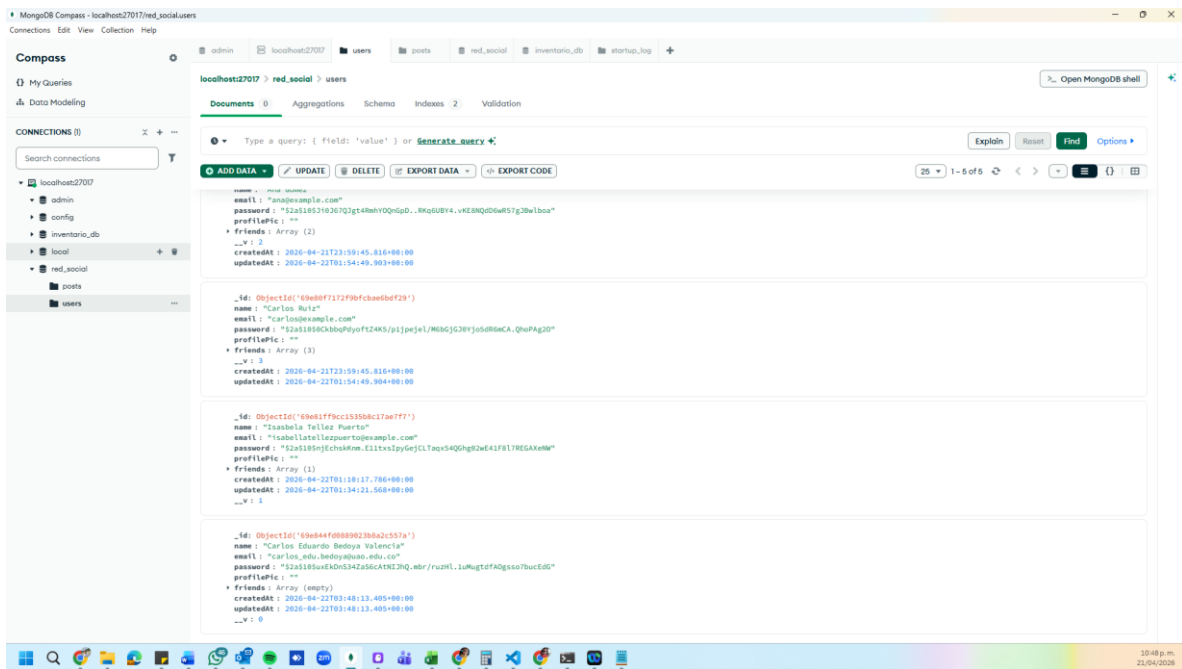
Registro de un usuario nuevo:

The screenshot shows a web browser window with the URL `localhost:3000/api/posts` and a tab titled 'Registro de Usuario'. The page displays a registration form titled 'Registro de Usuario'. The form includes a profile picture upload area with the text 'Seleccionar archivo' and 'Ningún archivo seleccionado'. Below this are input fields for the name 'Carlos Eduardo Bedoya Valencia', a password field with masked characters, and an email field containing 'carlos_edu.bedoya@uao.edu.co'. A red 'Registrarse' button is positioned below the email field. At the bottom of the form, there is a link that says '¿Ya tienes cuenta? Iniciar sesión'. The browser's taskbar at the bottom shows various application icons and the system clock indicating 10:47 p.m. on 21/04/2020.

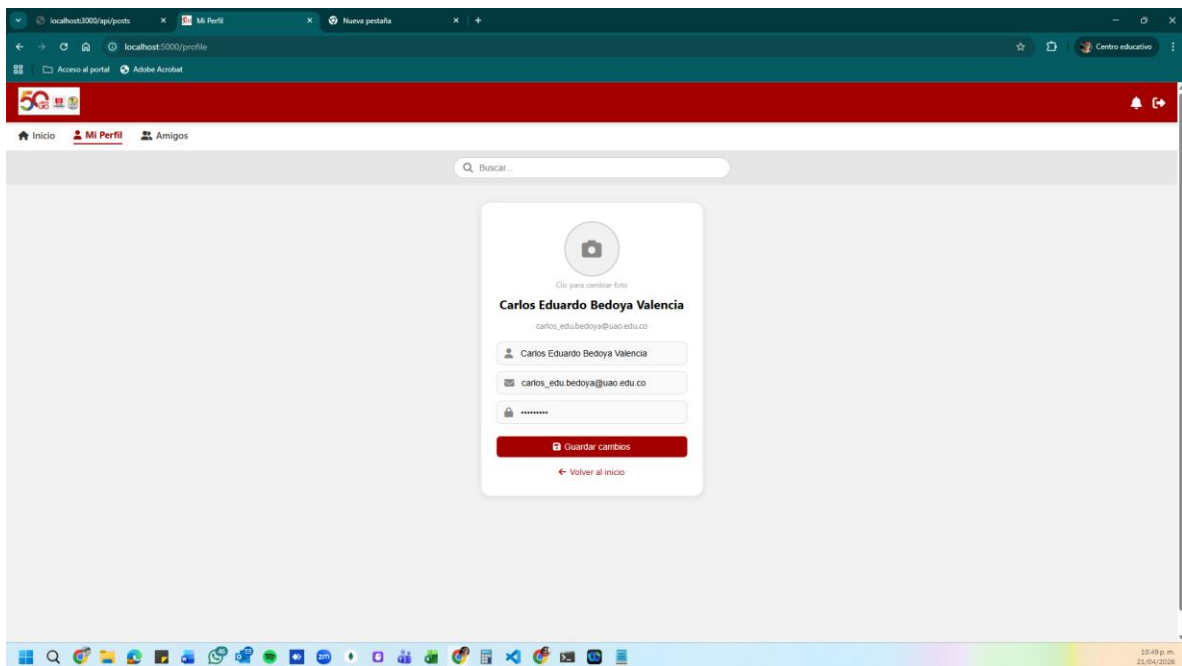
Perfil creado:

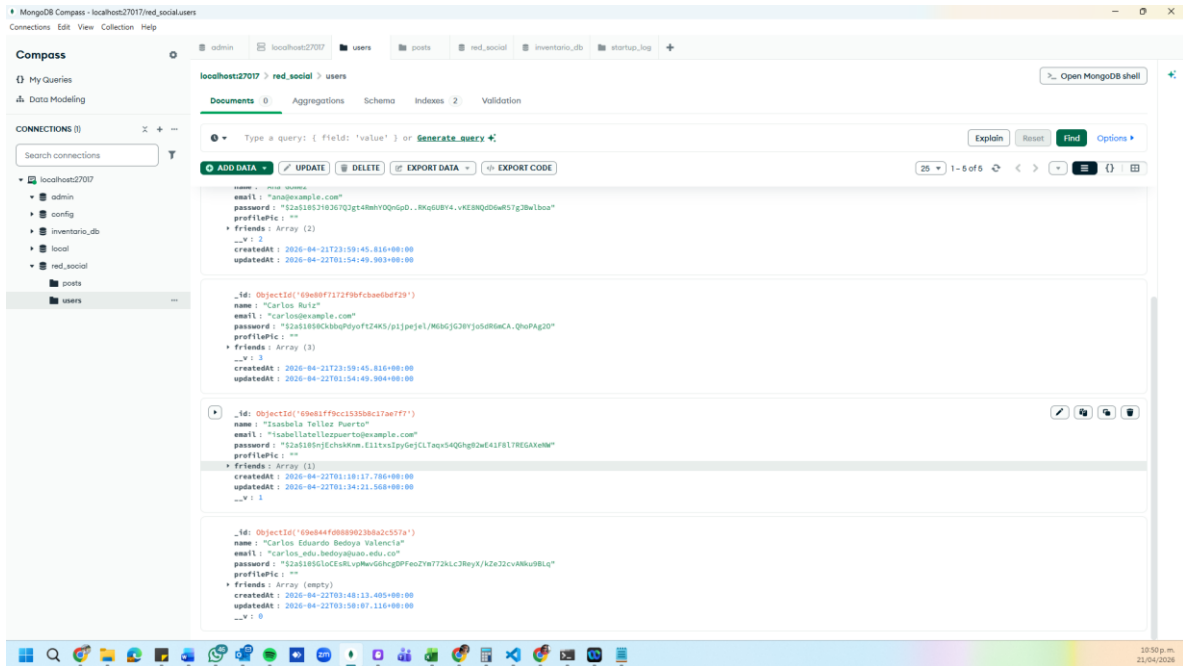
The screenshot shows the 'Mi Perfil' (My Profile) page in the same web browser. The URL is `localhost:3000/profile`. The page features a red header with a '50' anniversary logo and navigation links for 'Inicio', 'Mi Perfil', and 'Amigos'. A search bar is located below the header. The main content area displays the user's profile information: a profile picture placeholder with the text 'Clic para cambiar foto', the name 'Carlos Eduardo Bedoya Valencia', and the email 'carlos_edu.bedoya@uao.edu.co'. Below this are input fields for the name (pre-filled with 'Carlos Eduardo Bedoya Valencia') and email (pre-filled with 'carlos_edu.bedoya@uao.edu.co'), and a field for a new password with the label 'Nueva contraseña (opcional)'. A red 'Guardar cambios' button is at the bottom of the profile card, with a 'Volver al inicio' link below it. The browser's taskbar at the bottom shows the same application icons and system clock as the previous screenshot.

Registrado en la base de datos:

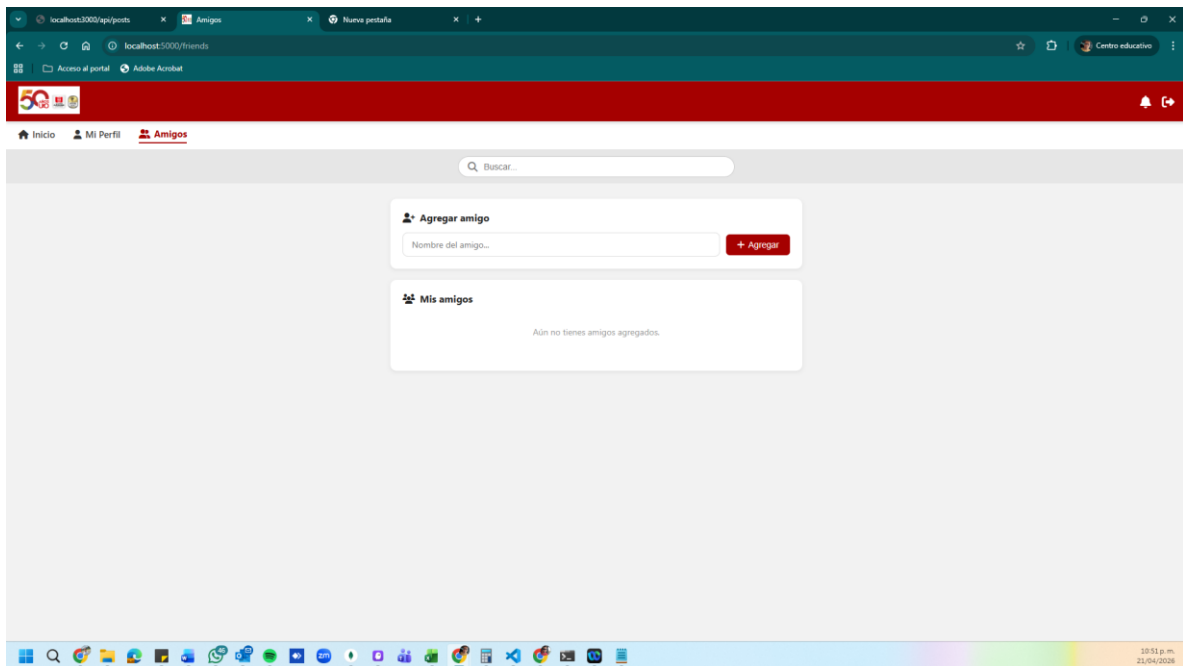


Cambio de contraseña y almacenando al información en la base de datos:

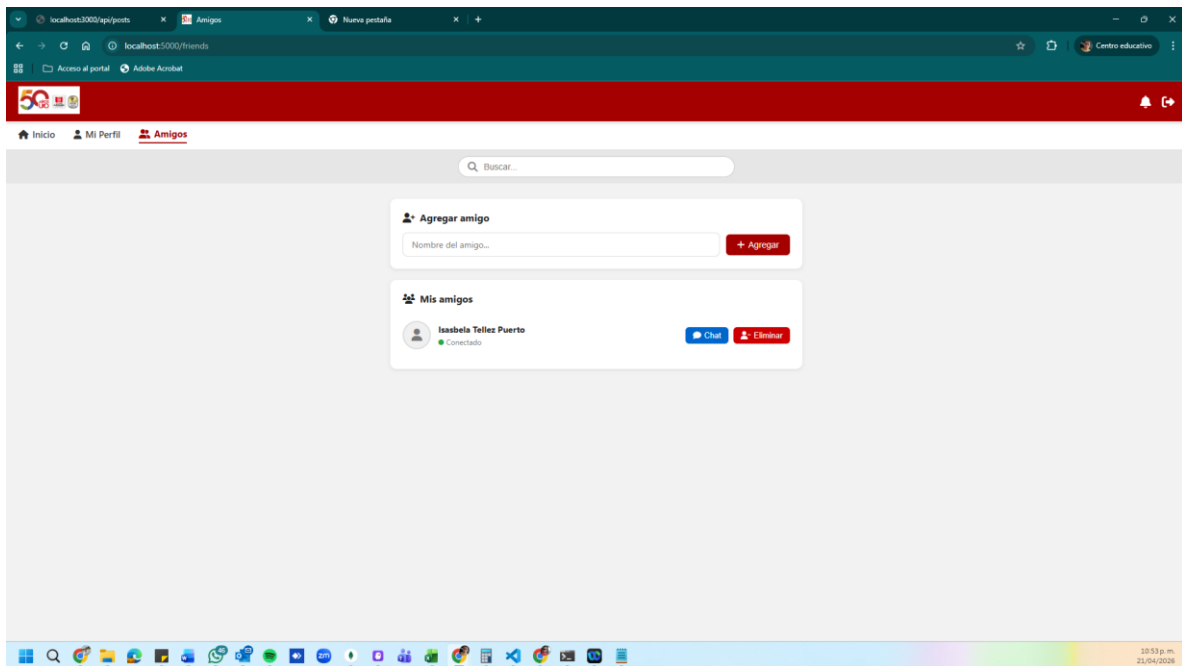
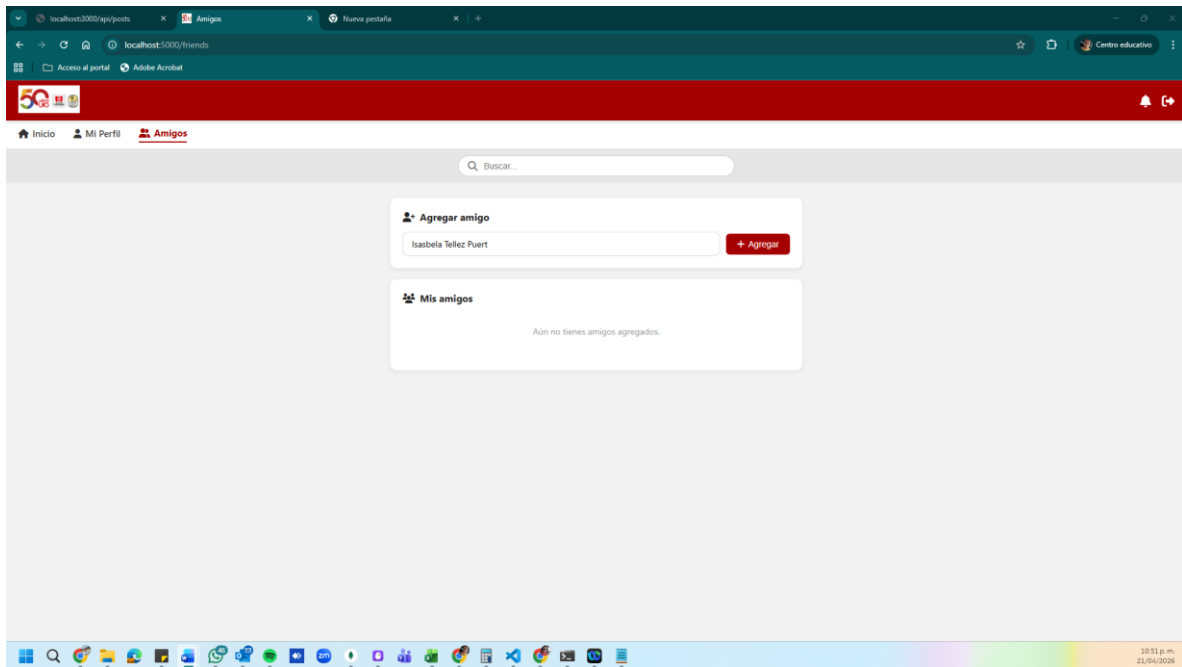




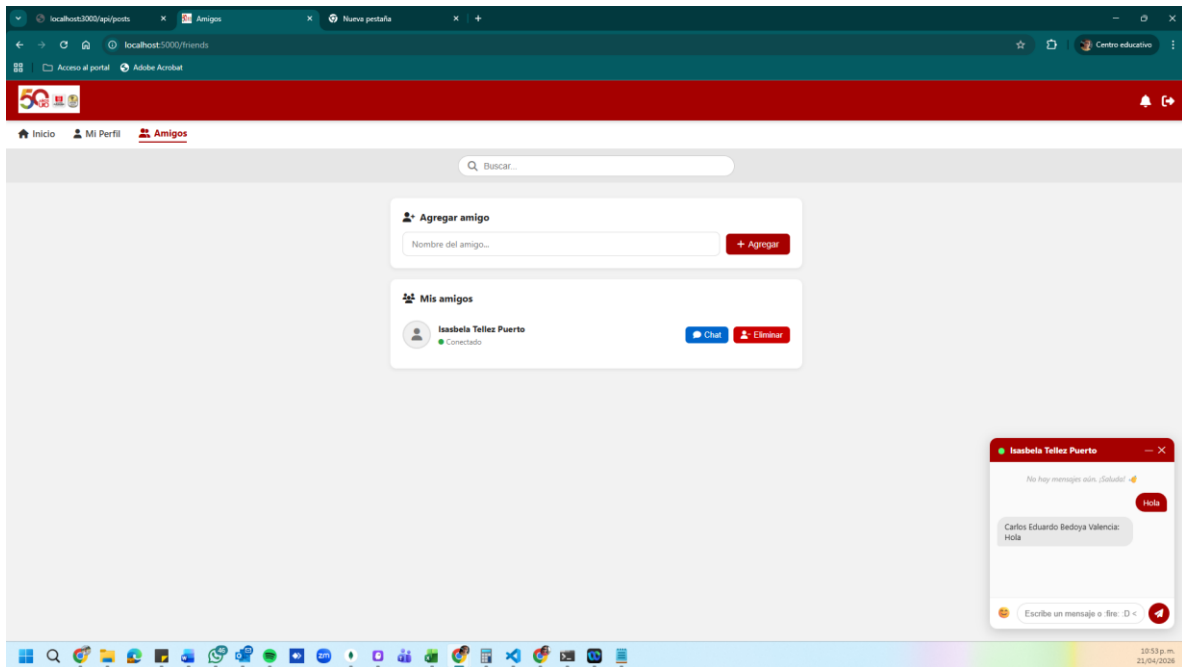
Perfil nuevo sin amigos:



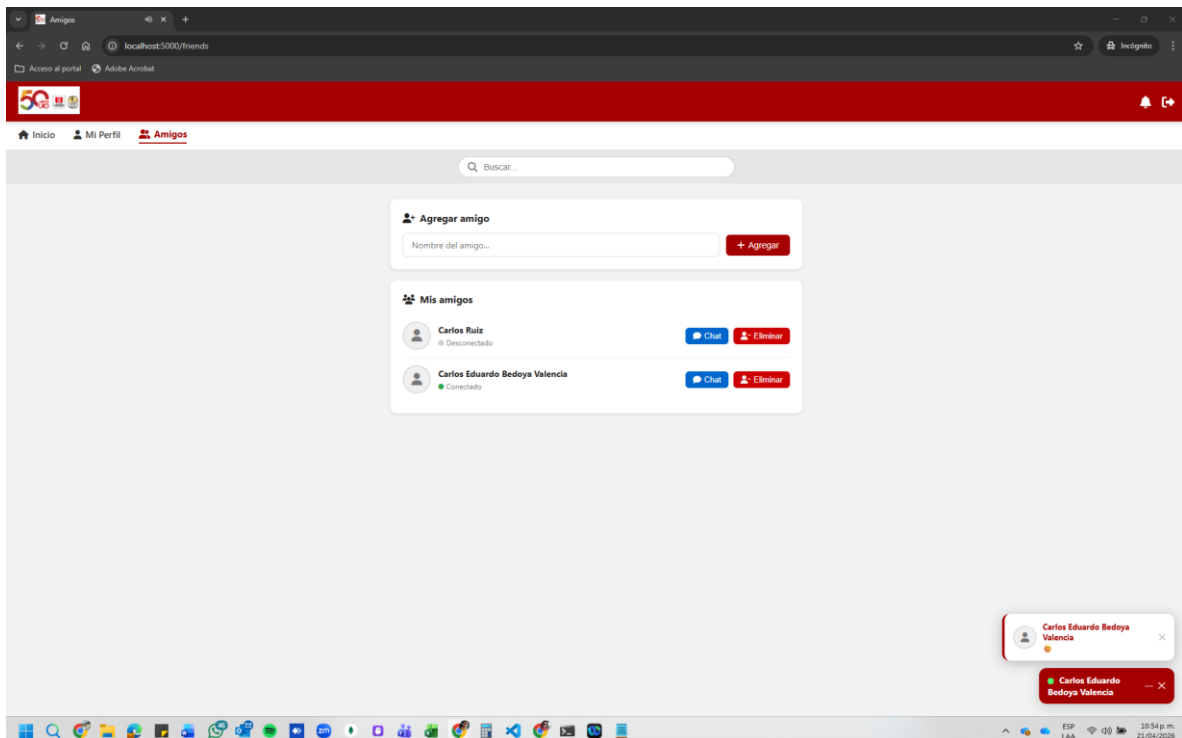
Permite agregar amigos:



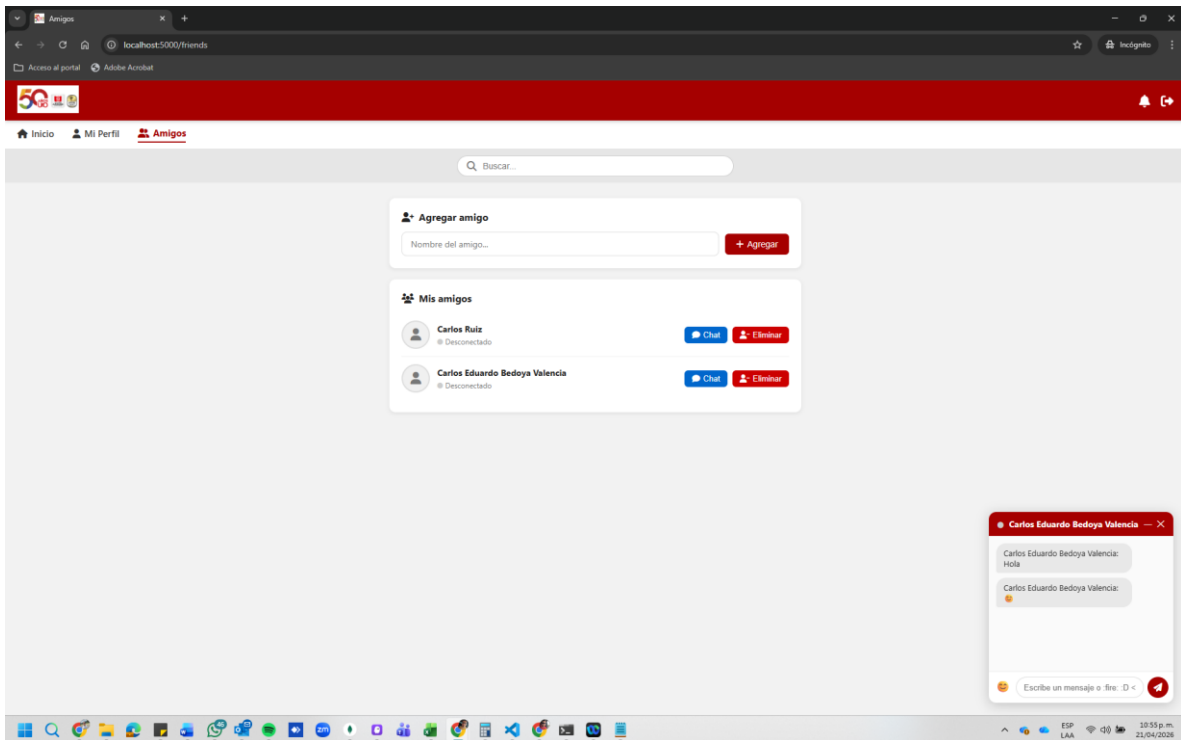
Chat en funcionamiento:



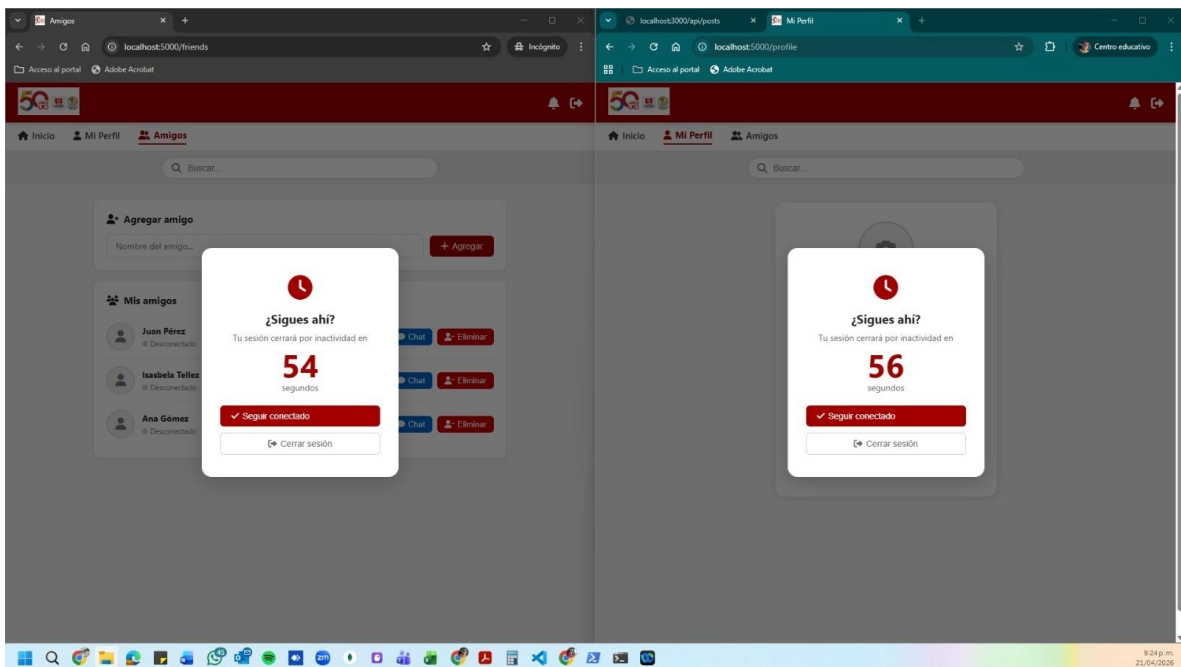
Con sus respectivas alertas y notificaciones:

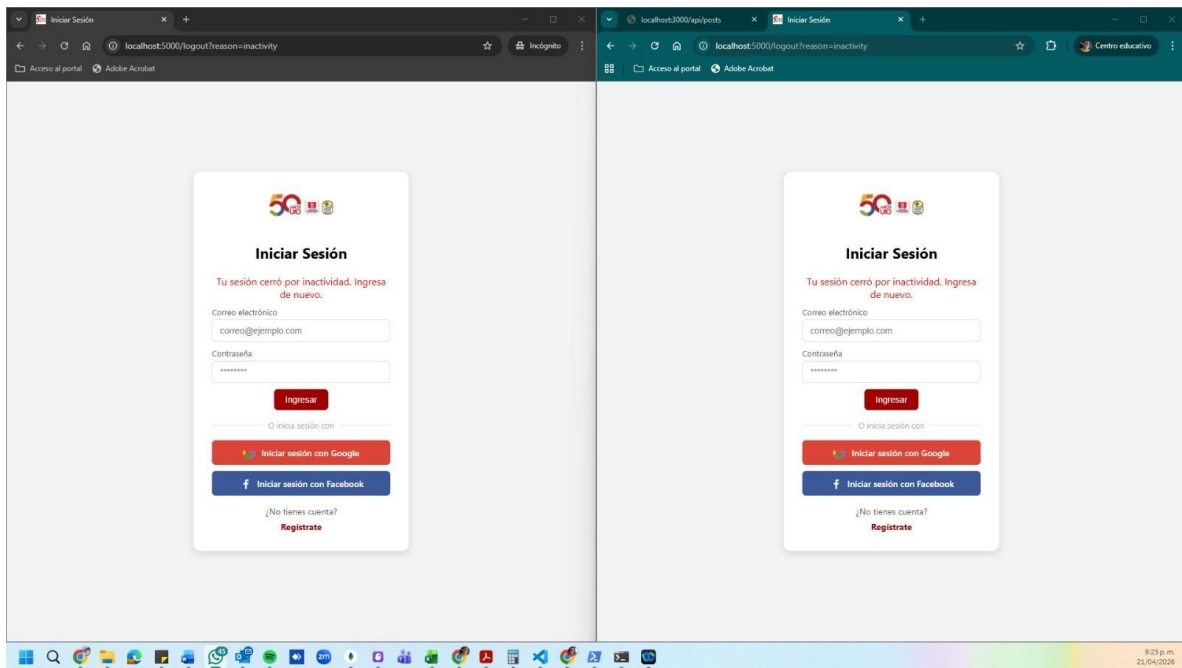


Permite saber si el usuario esta conectado o no está conectado:

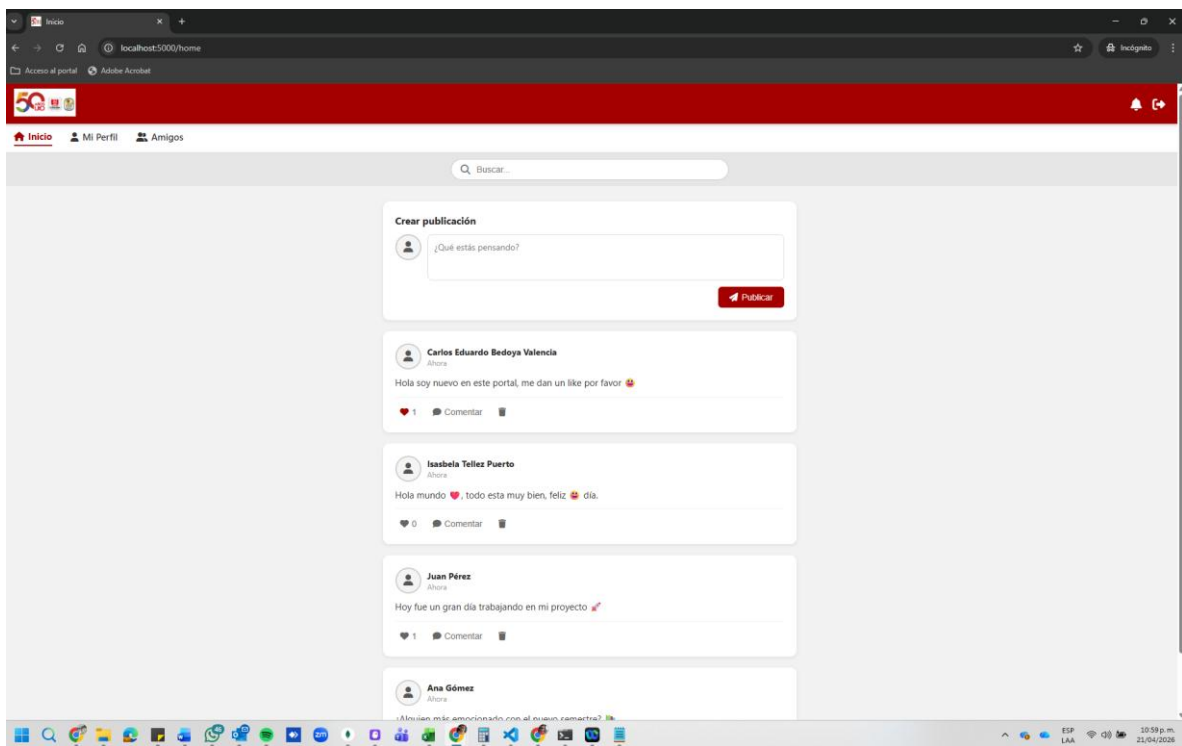


Implementación de regla de seguridad para cerrar la sección por inactividad, después de 4 minutos sin utilizar la plataforma:

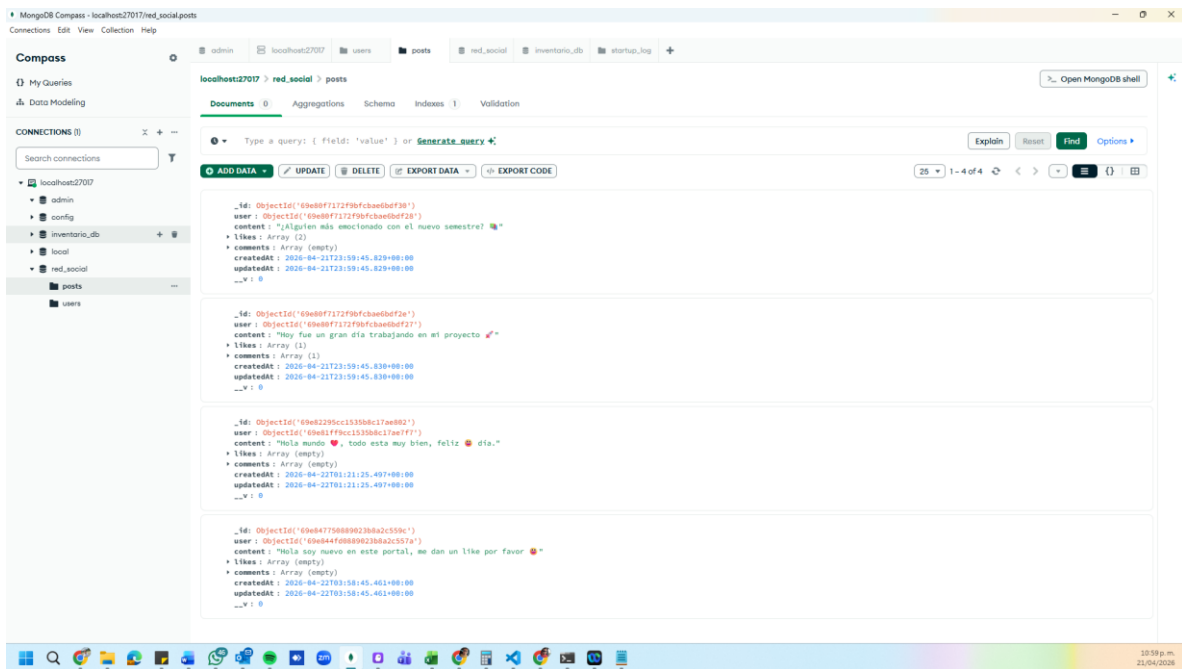




Publicaciones en tiempo real:



Almacenando todo en la base de datos:



Conclusión

En esta segunda fase del proyecto se logró consolidar una arquitectura backend completa, integrando servicios web, base de datos y comunicación en tiempo real. La implementación de una estructura modular permitió organizar eficientemente el sistema, facilitando su escalabilidad y mantenimiento.

El uso de MongoDB como sistema de persistencia proporcionó flexibilidad en el manejo de datos, mientras que la incorporación de sockets permitió enriquecer la experiencia del usuario mediante interacciones dinámicas. En conjunto, este avance representa un paso fundamental hacia el desarrollo de una plataforma social funcional y alineada con estándares modernos de desarrollo web.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

Flanagan, D. (2020). *JavaScript: The definitive guide* (7th ed.). O'Reilly Media.

MongoDB, Inc. (2024). *MongoDB documentation*. <https://www.mongodb.com/docs/>

Node.js Foundation. (2024). *Node.js documentation*. <https://nodejs.org/en/docs>

Express.js. (2024). *Express web framework documentation*. <https://expressjs.com/>

Mozilla Developer Network. (2024). *JavaScript documentation*. <https://developer.mozilla.org/>

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.